

TP - Découverte et Prise en Main de Docker

Objectif Pédagogique : Se familiariser avec les concepts de base de Docker (images, conteneurs) et manipuler les commandes essentielles pour exécuter et gérer des conteneurs.

Prérequis : Avoir Docker Desktop ou Docker Engine installé sur la machine.

Exercice 1 : Manipulation des images et lancement de conteneurs

1. Exécuter un premier conteneur avec `docker run hello-world`. Analysez de ce qui s'est passé.

```
C:\Users\bouya>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:d4aaab6242e0cace87e2ec17a2ed3d779d18fbfd03042ea58f2995626396a274
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

C:\Users\bouya>
```

La commande `docker run hello-world` télécharge l'image hello-world (si elle n'existe pas localement), crée un conteneur et l'exécute.

Le conteneur affiche un message confirmant que Docker est correctement installé, puis il s'arrête automatiquement.

2. Suivez les étapes suivantes pour lancer un conteneur Nginx et **donnez l'instruction relative à chaque étape.**

Télécharger l'image Nginx

`docker pull nginx` → récupère l'image Nginx depuis Docker Hub.

Lancer le conteneur Nginx

`docker run -d -p 80:80 --name nginx-container nginx` → démarre Nginx en arrière-plan et expose le port 80.

Vérifier l'exécution

`docker ps` → affiche les conteneurs en cours d'exécution.

Tester dans le navigateur

Accéder à `http://localhost` → affiche la page par défaut de Nginx.

3. Quels sont les images Nginx disponibles sur le Docker Hub via la ligne de commande

```
C:\Users\bouya>docker search nginx
NAME                                DESCRIPTION                                STARS    OFFICIAL
nginx                               Official build of Nginx.                  21108    [OK]
nginx/nginx-ingress                NGINX and NGINX Plus Ingress Controllers fo... 111
nginx/nginx-prometheus-exporter    NGINX Prometheus Exporter for NGINX and NGIN... 50
nginx/unit                         This repository is retired, use the Docker o... 66
nginx/nginx-ingress-operator       NGINX Ingress Operator for NGINX and NGINX P... 3
nginx/nginx-quic-qns               NGINX QUIC interop                        1
nginx/nginxaas-loadbalancer-kubernetes 1
nginx/unit-preview                 Unit preview features                     0
bitnami/nginx                      Bitnami Secure Image for nginx            201
bitnamicharts/nginx               Bitnami Helm chart for NGINX Open Source    2
ubuntu/nginx                       Nginx, a high-performance reverse proxy & we... 133
kasmweb/nginx                      An Nginx image based off nginx:alpine and in... 8
rancher/nginx                      2
linuxserver/nginx                 An Nginx container, brought to you by LinuxS... 233
dtagdevsec/nginx                  T-Pot Nginx                              0
vmware/nginx                       3
paketobuildpacks/nginx            0
chainguard/nginx                   Build, ship and run secure software with Cha... 5
gluufederation/nginx              A customized NGINX image containing a consu... 1
antrea/nginx                       Nginx server used for Antrea e2e testing    0
intel/nginx                       0
circleci/nginx                    This image is for internal use              2
cleanstart/nginx                  Secure by Design, Built for Speed, Hardened ... 0
docksal/nginx                     Nginx service image for Docksal            1
corpusops/nginx                   https://github.com/corpusops/docker-images/ 1

C:\Users\bouya>
```

4. Télécharger la dernière version de l'image Nginx

```
C:\Users\bouya>docker pull nginx:latest
latest: Pulling from library/nginx
5b5fa0b64d74: Pull complete
114e699da838: Pull complete
9ee60c6c0558: Pull complete
5b219a92f92a: Pull complete
1733a4cd5954: Pull complete
7382b41547b8: Pull complete
ee3a09d2248a: Pull complete
Digest: sha256:fb01117203ff38c2f9af91db1a7409459182a37c87cced5cb442d1d8fcc66d19
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

C:\Users\bouya>
```

5. Exécuter un conteneur avec l'image Nginx en mode détaché (-d) et mapper un port (-p).

```
C:\Users\bouya>docker run -d -p 8080:80 nginx
46fc8d51f3224ac0504cf9d4fd8c672f22d0bb830204b9bf2fb844f832dfdfc3

C:\Users\bouya>
```

6. Vérifier que le serveur web NGINX est accessible depuis le navigateur.

- Assurez-vous que le container NGINX reste en exécution avant d'exécuter les étapes suivantes

```
C:\Users\bouya>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
46fc8d51f322   nginx     "/docker-entrypoint..." About a minute ago Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
hopeful_meitner
```

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

7. Lister les images existantes dans votre hôte.

```
C:\Users\bouya>docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx          latest    fb01117203ff   3 days ago    225MB
hello-world    latest    d4aaab6242e0   4 months ago  20.3kB
```

8. Utiliser la commande *docker system prune* pour nettoyer les ressources inutilisées (conteneurs arrêtés, réseaux non utilisés, images non référencées, build caches).

```
C:\Users\bouya>docker system prune
WARNING! This will remove:
 - all stopped containers
 - all networks not used by at least one container
 - all dangling images
 - unused build cache

Are you sure you want to continue? [y/N] y
Deleted Containers:
7bf8eff59003e212e98b7b0dddc510f9145591831691f7e8f6d6d10aa4b29119
27049dac34a7deafc4160fb43e28f6e66add0cd4d5b424e2ccd9bb8df4065807

Deleted build cache objects:
0igLto1ii1scqi58vsgzua11n
rgd00ii81xkxshnbcrtfw670h
vegbwngutlbca4evewuhm1hgh
qb6ts8slhacbew3eom2d2ma5d
bh6kgx4y93umk293o95bi8c0f
hgff2ro2zp6j8drmmvouwuivm
iehrmi0f4ulxtorhobhnsna5v
tnuwmvwpwcqgy7z9msgjjwpm
k1wfvb7433h9l93vbxpzdht2v
9fbl8mn93pnbcnuol9boeil2
k88z8nf8b51ci5x6kik5vybl2
y21pyz4e9sjqg6182cm97hrht
kd895ht43u2fm190setf23kua
vpmxxpqg0xzqmth7uzxfp9qau

Total reclaimed space: 1.244GB

C:\Users\bouya>
```

9. Lister les images existantes dans votre hôte. Expliquez pourquoi il y a des images qui ont disparu et d'autres non.

```
C:\Users\bouya>docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    fb01117203ff   3 days ago    225MB
hello-world    latest    d4aaab6242e0   4 months ago   20.3kB

C:\Users\bouya>
```

Certaines images ont disparu car docker system prune supprime les images **non utilisées** (sans conteneur associé). Les images encore présentes sont celles **toujours utilisées par des conteneurs en cours d'exécution** ou **encore référencées**.

Exercice 2 : Gestion des conteneurs

- Lancer 3 containers *container1*, *container2*, *container3* en utilisant l'image NGINX

```
C:\Users\bouya>docker run -d -p 8081:80 --name container1 nginx
18a7e9c0663007adc412d32e49ec15ddea7de4fd1cc2ef2b727ebc9609625fed

C:\Users\bouya>docker run -d -p 8082:80 --name container2 nginx
4fd3e30135fd8a29a995617a9fc1846c8a10dca230dd3c0675698d5c3742a4f1

C:\Users\bouya>docker run -d -p 8083:80 --name container3 nginx
8fb1e4cf0196a939a7afa6f6e9a9246423749b389b0ce3cfeaa601e37a52669f

C:\Users\bouya>
```

1. Lister tous les conteneurs (quel que soit leurs états).

```
C:\Users\bouya>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
8fb1e4cf0196   nginx    "/docker-entrypoint..." 51 seconds ago Up 51 seconds 0.0.0.0:8083->80/tcp, [::]:8083->80/tcp  container3
4fd3e30135fd   nginx    "/docker-entrypoint..." 53 seconds ago Up 52 seconds 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp  container2
18a7e9c06630   nginx    "/docker-entrypoint..." 54 seconds ago Up 54 seconds 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  container1
46fc8d51f322   nginx    "/docker-entrypoint..." 2 hours ago    Up 2 hours    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  hopeful_meitner

C:\Users\bouya>
```

2. Inspecter Container1 pour obtenir ses informations détaillées (docker inspect).

```
C:\Users\bouya>docker inspect container1
[
  {
    "Id": "18a7e9c0663007adc412d32e49ec15ddea7de4fd1cc2ef2b727ebc9609625fed",
    "Created": "2025-12-13T12:31:14.468244989Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 1613,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2025-12-13T12:31:14.761545791Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:fb01117203ff38c2f9af91db1a7409459182a37c87cced5cb442d1d8fcc66d19",
    "ResolvConfPath": "/var/lib/docker/containers/18a7e9c0663007adc412d32e49ec15ddea7de4fd1cc2ef2b727ebc9609625fed/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/18a7e9c0663007adc412d32e49ec15ddea7de4fd1cc2ef2b727ebc9609625fed/hostname",
    "HostsPath": "/var/lib/docker/containers/18a7e9c0663007adc412d32e49ec15ddea7de4fd1cc2ef2b727ebc9609625fed/hosts",
    "LogPath": "/var/lib/docker/containers/18a7e9c0663007adc412d32e49ec15ddea7de4fd1cc2ef2b727ebc9609625fed/18a7e9c0663007adc412d32e49ec15ddea7de4fd1cc2ef2b727ebc9609625fed-json.log",
    "Name": "/container1",
    "RestartCount": 0,
  }
]
```

- Donnez l'adresse IP du conteneur Container1 et Container2 ? Utilisez la commande `docker inspect` combiné avec l'option `--format` pour extraire directement cette information du JSON.

- `docker inspect --format '{{ IPAddress: "172.17.0.3 }}" }' container1`

Conteneur 1 "IPAddress": "172.17.0.3",

Conteneur 2 "IPAddress": "172.17.0.4",

- Connectivité des conteneurs :

- À quels réseaux appartiennent Container1 et Container2 ? Donnez l'ID

"Gateway": "172.17.0.1",

- Est-ce que les deux conteneurs Container1 et Container2 peuvent communiquer ? accéder à un des conteneurs et faites un ping (installer le ping dans le conteneur si nécessaire).

```

root@18a7e9c06630:/# ping 172.17.0.4
PING 172.17.0.4 (172.17.0.4) 56(84) bytes of data.
64 bytes from 172.17.0.4: icmp_seq=1 ttl=64 time=5.48 ms
64 bytes from 172.17.0.4: icmp_seq=2 ttl=64 time=0.118 ms
64 bytes from 172.17.0.4: icmp_seq=3 ttl=64 time=0.124 ms
64 bytes from 172.17.0.4: icmp_seq=4 ttl=64 time=0.124 ms
64 bytes from 172.17.0.4: icmp_seq=5 ttl=64 time=0.126 ms
64 bytes from 172.17.0.4: icmp_seq=6 ttl=64 time=0.116 ms
64 bytes from 172.17.0.4: icmp_seq=7 ttl=64 time=0.129 ms
64 bytes from 172.17.0.4: icmp_seq=8 ttl=64 time=0.109 ms
64 bytes from 172.17.0.4: icmp_seq=9 ttl=64 time=0.131 ms
64 bytes from 172.17.0.4: icmp_seq=10 ttl=64 time=0.126 ms
64 bytes from 172.17.0.4: icmp_seq=11 ttl=64 time=0.105 ms
64 bytes from 172.17.0.4: icmp_seq=12 ttl=64 time=0.100 ms
64 bytes from 172.17.0.4: icmp_seq=13 ttl=64 time=0.103 ms
64 bytes from 172.17.0.4: icmp_seq=14 ttl=64 time=0.115 ms
64 bytes from 172.17.0.4: icmp_seq=15 ttl=64 time=0.125 ms
64 bytes from 172.17.0.4: icmp_seq=16 ttl=64 time=0.100 ms
64 bytes from 172.17.0.4: icmp_seq=17 ttl=64 time=0.145 ms
64 bytes from 172.17.0.4: icmp_seq=18 ttl=64 time=0.119 ms
64 bytes from 172.17.0.4: icmp_seq=19 ttl=64 time=0.191 ms
64 bytes from 172.17.0.4: icmp_seq=20 ttl=64 time=0.134 ms
64 bytes from 172.17.0.4: icmp_seq=21 ttl=64 time=0.104 ms
64 bytes from 172.17.0.4: icmp_seq=22 ttl=64 time=0.125 ms
64 bytes from 172.17.0.4: icmp_seq=23 ttl=64 time=0.101 ms

```

les conteneurs peuvent communiquer sur le réseau **bridge**.

6. Est-ce que Container1 a accès à Internet ? (Assurez-vous que l'hôte a accès à Internet avant de faire vos tests).

```

root@18a7e9c06630:/# ping -c 4 google.com
PING google.com (74.125.29.113) 56(84) bytes of data.
64 bytes from 74.125.29.113: icmp_seq=1 ttl=63 time=115 ms
64 bytes from 74.125.29.113: icmp_seq=2 ttl=63 time=64.5 ms
64 bytes from 74.125.29.113: icmp_seq=3 ttl=63 time=53.6 ms
64 bytes from 74.125.29.113: icmp_seq=4 ttl=63 time=269 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 53.634/125.417/268.540/85.820 ms
root@18a7e9c06630:/#

```

7. Consulter les logs de container2 (docker logs). Qu'est-ce qu'il montre ? expliquez donc le rôle de la commande log.

```

C:\Users\bouya>docker logs container2
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/12/13 12:31:17 [notice] 1#1: using the "epoll" event method
2025/12/13 12:31:17 [notice] 1#1: nginx/1.29.4
2025/12/13 12:31:17 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2025/12/13 12:31:17 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2025/12/13 12:31:17 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2025/12/13 12:31:17 [notice] 1#1: start worker processes
2025/12/13 12:31:17 [notice] 1#1: start worker process 30
2025/12/13 12:31:17 [notice] 1#1: start worker process 31
2025/12/13 12:31:17 [notice] 1#1: start worker process 32
2025/12/13 12:31:17 [notice] 1#1: start worker process 33
2025/12/13 12:31:17 [notice] 1#1: start worker process 34
2025/12/13 12:31:17 [notice] 1#1: start worker process 35
2025/12/13 12:31:17 [notice] 1#1: start worker process 36
2025/12/13 12:31:17 [notice] 1#1: start worker process 37

C:\Users\bouya>

```

- Système de fichier d'un container :

Chaque conteneur a son propre **système de fichiers isolé**, basé sur l'image.

Les modifications sont **éphémères** sauf si des volumes sont utilisés pour la persistance des données.

8. Qu'est-ce qu'un système de fichier d'un conteneur ?

Le système de fichiers d'un conteneur est un espace isolé qui contient tous les fichiers nécessaires à l'exécution du conteneur, basé sur son image Docker.

- Chaque conteneur a son propre système de fichiers indépendant des autres conteneurs et de l'hôte.
- Les modifications apportées à ce système sont éphémères à moins d'utiliser des volumes pour la persistance des données.

9. Accéder au container2 et créer dedans le fichier « exemple.txt » qui contient le texte suivant « bienvenu »

```

C:\Users\bouya>docker exec -it container2 bash
root@4fd3e30135fd:/# echo "bienvenu" > exemple.txt
root@4fd3e30135fd:/#

```

10. Arrêtez Container1 et container2 et vérifier qu'ils sont arrêtés et que le container3 est encore en cours d'exécution. Quelle est la commande qui permet de vérifier l'état des containers. Donner son output.

```

C:\Users\bouya>docker exec -it container2 bash
root@4fd3e30135fd:/# echo "bienvenu" > exemple.txt
root@4fd3e30135fd:/# exit
exit

C:\Users\bouya>docker stop container1 container2
container1
container2

C:\Users\bouya>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
NAMES
8fb1e4cf0196   nginx    "/docker-entrypoint..." 38 minutes ago Up 38 minutes  0.0.0.0:8083->80/tcp, [::]:8083->80/tcp
p   container3
4fd3e30135fd   nginx    "/docker-entrypoint..." 38 minutes ago Exited (0) 28 seconds ago
container2
18a7e9c06630   nginx    "/docker-entrypoint..." 38 minutes ago Exited (0) 28 seconds ago
container1
46fc8d51f322   nginx    "/docker-entrypoint..." 3 hours ago    Up 3 hours    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
p   hopeful_meitner

C:\Users\bouya>

```

STATUS montre que **container1** et **container2** sont **arrêtés**, tandis que **container3** est **toujours en cours d'exécution**.

11. Redémarrer le container 2.

```
docker start container2
```

12. Est-ce que le fichier exemple.txt a été perdu ?

```

C:\Users\bouya>docker start container2
container2

C:\Users\bouya>docker exec -it container2 bash
root@4fd3e30135fd:/# ls
bin    dev          docker-entrypoint.sh  exemple.txt  lib    media  opt    root  sbin  sys  usr
boot  docker-entrypoint.d  etc                  home        lib64  mnt    proc  run   srv   tmp  var
root@4fd3e30135fd:/# |

```

Non, le fichier **exemple.txt** n'a pas été perdu.

13. Supprimer complètement container2 et vérifier qu'il n'est plus dans la liste des conteneurs

```

C:\Users\bouya>docker rm container2
container2

C:\Users\bouya>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
8fb1e4cf0196   nginx    "/docker-entrypoint..." 2 hours ago    Exited (0) 35 seconds ago        container3
18a7e9c06630   nginx    "/docker-entrypoint..." 2 hours ago    Exited (0) About an hour ago        container1
46fc8d51f322   nginx    "/docker-entrypoint..." 4 hours ago    Exited (0) 35 seconds ago        hopeful_meitner

C:\Users\bouya>

```

14. Est-ce que c'est possible de retrouver le fichier « exemple.txt » ?

Non, **il n'est plus possible de retrouver exemple.txt** après la suppression du conteneur.

15. Quelle est donc votre conclusion concernant la persistance du système de fichier du conteneur lorsque le conteneur est arrêté ? et lorsque le conteneur est complètement supprimé ?

- **Lorsque le conteneur est arrêté** : le système de fichiers **reste intact**, donc les fichiers créés sont conservés.
- **Lorsque le conteneur est supprimé** : le système de fichiers est **perdu**, et tous les fichiers créés à l'intérieur disparaissent.

Pour conserver les données, il faut utiliser **des volumes ou des dossiers mappés de l'hôte**.